

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Coding Challenge)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO2-AT2-CODING CHALLENGE
Question	Q2 — Challenge 2: Vowel and Consonant Classifier with Statistics (Easy 10 Marks)

Challenge 2: Vowel and Consonant Classifier with Statistics [Level: Easy | 10 Marks]

Accept a sentence from the user and display: (a) total vowels and consonants, (b) the most frequent vowel, (c) the least frequent consonant, and (d) the ratio of vowels to consonants as a decimal rounded to 2 places.

Constraints

- Case-insensitive analysis.
- Ignore spaces and digits.
- Handle sentences with no vowels or no consonants gracefully.

Skills Assessed

- Character classification with conditionals
- Counting and comparison logic
- Arithmetic and rounding

Sample Input / Output

Sample Input:
Python Programming

Expected Output:
Vowels: 5 | Consonants: 12
Most frequent vowel: o (2)
Least frequent consonant: y (1)
Vowel-to-Consonant Ratio: 0.42

Note on the answer key: the totals Vowels: 5 / Consonants: 12 for “Python Programming” (17 letters) only add up if ‘y’ is counted as a vowel (the a/e/i/o/u/y convention). This solution adopts that rule, which reproduces the given vowel/consonant totals, the most-frequent-vowel result, and the ratio exactly. Since ‘y’ is then already counted as a vowel, it cannot consistently also be reported as the least frequent

consonant — among the remaining consonants both 't' and 'h' occur once, so the program reports 't', the first one encountered. This discrepancy is explained further in the Sample Output section.

Approach / Design

- **Cleaning (get_letters_only):** lowercases the sentence and keeps only alphabetic characters via `isalpha()`, automatically ignoring spaces, digits, and punctuation.
- **Classification (classify_letters):** loops through the cleaned letters and uses a conditional membership check against a vowel set (a, e, i, o, u, y) to route each letter into a vowel-count or consonant-count dictionary.
- **Most/least frequent (find_most_frequent / find_least_frequent):** each loops through a counts dictionary using explicit comparison logic (rather than built-in `max()/min()`) to track the highest or lowest count, with the least-frequent finder keeping the first-encountered letter on a tie.
- **Ratio (calculate_ratio):** divides vowel total by consonant total and rounds to 2 decimal places, returning `None` (handled gracefully) when there are no consonants to avoid a division-by-zero error.
- **Full analysis (analyze_sentence):** orchestrates cleaning, classification, frequency lookups, and ratio calculation into one summary dictionary, working correctly even when there are no vowels or no consonants at all.
- **Reporting (print_report):** prints all four required statistics, substituting graceful messages when a vowel or consonant category is empty.

Python Program

```
"""
C02-AT2 - Coding Challenge
Challenge 2: Vowel and Consonant Classifier with Statistics
[Level: Easy | 10 Marks]
-----
Accepts a sentence from the user and displays:
(a) total vowels and consonants
(b) the most frequent vowel
(c) the least frequent consonant
(d) the vowel-to-consonant ratio as a decimal rounded to 2 places

Design note on the sample answer key:
The sample totals "Vowels: 5 | Consonants: 12" for the input
"Python Programming" only add up (5 + 12 = 17 letters) if 'y' is
counted as a vowel. Treating 'y' as a vowel is a common convention
(a/e/i/o/u/y), so that is the rule used below, and it reproduces
the sample's vowel/consonant totals and most-frequent-vowel result
exactly. With 'y' already counted as a vowel, it cannot also be the
"least frequent consonant" -- among the remaining consonants
P,t,h,n,P,r,g,r,m,m,n,g, both 't' and 'h' occur exactly once, so
this program reports the first one encountered ('t').
"""

VOWELS = set("aeiouy")

# -----
# 1. Cleaning helper -- keep only alphabetic characters
# -----
def get_letters_only(sentence):
    """
    Returns a lowercased list of characters from the sentence,
    ignoring spaces, digits, and punctuation -- keeping letters
    only (case-insensitive analysis).
    """
    letters = []
    for char in sentence.lower():
        if char.isalpha():
            letters.append(char)
```

```

        if char.isalpha():
            letters.append(char)
    return letters

# -----
# 2. Classification function
# -----
def classify_letters(letters):
    """
    Loops through the letters and classifies each as a vowel or
    consonant using a conditional membership check, counting
    occurrences of each individual letter along the way.
    """
    vowel_counts = {}
    consonant_counts = {}

    for char in letters:
        if char in VOWELS:
            vowel_counts[char] = vowel_counts.get(char, 0) + 1
        else:
            consonant_counts[char] = consonant_counts.get(char, 0) + 1

    return vowel_counts, consonant_counts

# -----
# 3. Most / least frequent letter finder (explicit comparison loop)
# -----
def find_most_frequent(counts):
    """
    Finds the letter with the highest count using a loop and
    comparison logic. Returns (letter, count) or (None, 0) if
    counts is empty.
    """
    top_letter, top_count = None, 0
    for letter, count in counts.items():
        if count > top_count:
            top_letter, top_count = letter, count
    return top_letter, top_count

def find_least_frequent(counts):
    """
    Finds the letter with the lowest count using a loop and
    comparison logic. On ties, the first letter encountered
    (dictionary insertion order) is kept. Returns (letter, count)
    or (None, 0) if counts is empty.
    """
    least_letter, least_count = None, None
    for letter, count in counts.items():
        if least_count is None or count < least_count:
            least_letter, least_count = letter, count
    return least_letter, (least_count or 0)

# -----
# 4. Ratio calculation function
# -----
def calculate_ratio(vowel_total, consonant_total):
    """
    Calculates the vowel-to-consonant ratio as a decimal rounded
    to 2 places. Handles the case where there are no consonants
    (division by zero) gracefully.
    """
    if consonant_total == 0:
        return None
    return round(vowel_total / consonant_total, 2)
# -----

```

```

# 5. Full analysis function
# -----
def analyze_sentence(sentence):
    """
    Orchestrates cleaning, classification, frequency analysis,
    and ratio calculation, returning a summary dictionary. Handles
    sentences with no vowels or no consonants gracefully.
    """
    letters = get_letters_only(sentence)
    vowel_counts, consonant_counts = classify_letters(letters)

    vowel_total = sum(vowel_counts.values())
    consonant_total = sum(consonant_counts.values())

    most_frequent_vowel = find_most_frequent(vowel_counts) if vowel_counts else (None, 0)
    least_frequent_consonant = find_least_frequent(consonant_counts) if consonant_counts else (None, 0)

    ratio = calculate_ratio(vowel_total, consonant_total)

    return {
        "vowel_total": vowel_total,
        "consonant_total": consonant_total,
        "most_frequent_vowel": most_frequent_vowel,
        "least_frequent_consonant": least_frequent_consonant,
        "ratio": ratio,
    }

# -----
# 6. Report/display function
# -----
def print_report(result):
    print(f"Vowels: {result['vowel_total']} | Consonants: {result['consonant_total']}")

    if result["most_frequent_vowel"][0] is not None:
        letter, count = result["most_frequent_vowel"]
        print(f"Most frequent vowel: {letter} ({count})")
    else:
        print("Most frequent vowel: none found")

    if result["least_frequent_consonant"][0] is not None:
        letter, count = result["least_frequent_consonant"]
        print(f"Least frequent consonant: {letter} ({count})")
    else:
        print("Least frequent consonant: none found")

    if result["ratio"] is not None:
        print(f"Vowel-to-Consonant Ratio: {result['ratio']}")
    else:
        print("Vowel-to-Consonant Ratio: undefined (no consonants found)")

# -----
# 7. Main driver
# -----
def main():
    sentence = input("Enter a sentence: ")
    result = analyze_sentence(sentence)
    print_report(result)

if __name__ == "__main__":
    main()

```

Sample Output

```
Enter a sentence: Python Programming
Vowels: 5 | Consonants: 12
Most frequent vowel: o (2)
Least frequent consonant: t (1)
Vowel-to-Consonant Ratio: 0.42
```

Note: The consonants 't' and 'h' are tied at 1 occurrence each; the program reports the first one encountered ('t'). The provided answer key lists 'y (1)' here, but 'y' is already counted among the 5 vowels above (needed to make Vowels + Consonants = 17 match the given totals), so it cannot also be the least-frequent consonant in a consistent design.

Conclusion

The program classifies each letter of the input sentence as a vowel or consonant using a set-membership conditional, tracks per-letter frequencies in two dictionaries, and finds the most/least frequent letter in each category using explicit comparison loops rather than built-in shortcuts. The vowel-to-consonant ratio is computed with `round()` to two decimal places, and a division-by-zero guard handles sentences with no consonants (or no vowels) gracefully. Running the program on “Python Programming” reproduces the sample's vowel/consonant totals, most frequent vowel, and ratio exactly; the one difference (least frequent consonant) is explained above.